The Singular Value Decomposition

We've seen how useful matrix factorization can be for solving linear systems. We can solve square systems using the LU (or PLU) decomposition with backward and forward substitution. The QR factorization allows us to take advantage of an orthogonal matrix Q to solve even rectangular systems with a single backward substitution. We will now discuss one final matrix factorization: the singular value decomposition.

We have seen that orthonormal matrices are very convenient because we can find the inverse by simply taking the transpose. Another class of matrix that is easily invertible is the diagonal matrix.

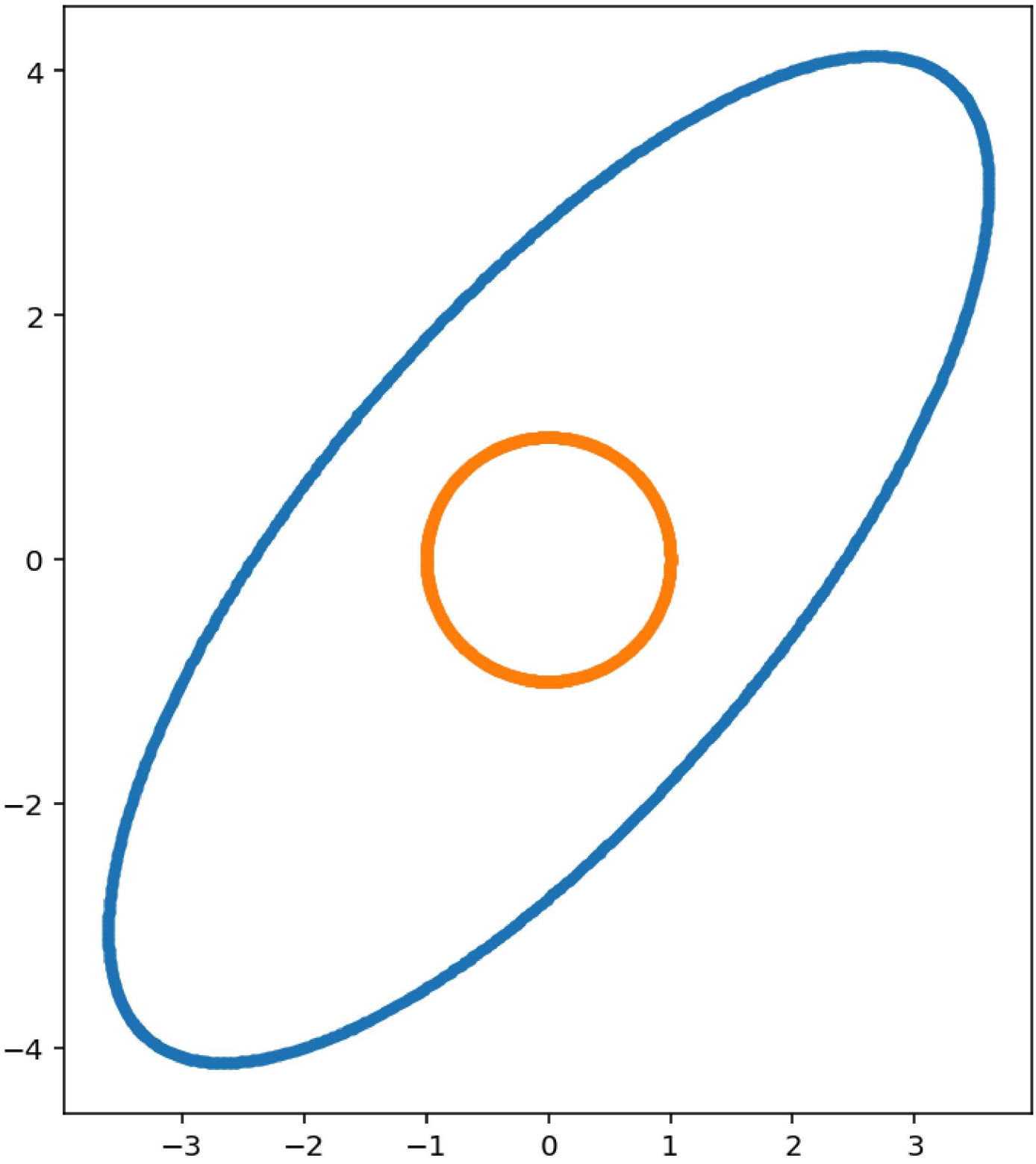
Exercise 1. Consider an m by m diagonal matrix where the diagonal entries are σ_1 through σ_n. What is the inverse of this matrix?

It would be extraordinarily convenient if we could express a matrix as a product of orthogonal and diagonal matrices. Somewhat shockingly, we can! The singular value decomposition allows us to write ***any*** m x n matrix A as the product of three matrices:

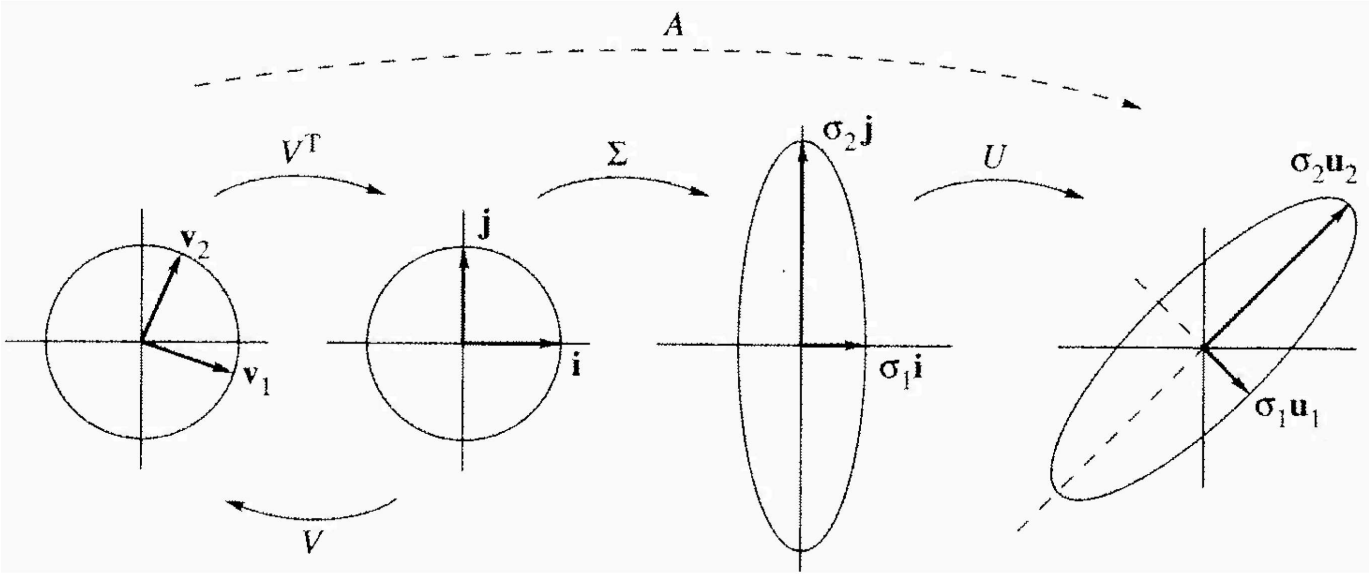
$$A = U\Sigma V^T$$

Where U is an m x m orthonormal matrix, V is an n x n orthonormal matrix, and Σ is an m x n diagonal matrix with the entries organized in decreasing magnitude. Let's get some geometric intuition behind how this works. Orthonormal matrices can be thought of as rotations and diagonal matrices can be thought of as scaling along the standard basis of a space. All linear transforms (and thus, matrices) are simple combinations of rotations and scaling.

Consider a 2 x 2 matrix A for simplicity. When the matrix A is applied to any unit vector, that vector is rotated and stretched. Consider applying A to many unit vectors, all arranged on the unit circle. What happens? Below in orange are 1000 equally spaced points around the unit circle. I multiplied each by the matrix $A = \begin{bmatrix} 3 & -2 \\ 1 & -4 \end{bmatrix}$. The results are plotted in blue.



It turns the unit circle into an ellipse! In fact, this is generically true. If you have an m x n matrix A and apply it to the n-dimensional unit ball, you will end up with an m-dimensional hyperellipsoid. There is a deep connection to the SVD here. The directions of the axes of that hyper ellipsoid form the columns of the matrix U in the SVD. The magnitude of those axes are the diagonal entries of Σ. And the columns of V are what specify which directions in the original unit ball get mapped to the ellipsoid axes (aka the preimages of the columns of U).



Before we see how to find this decomposition, let's see how it could be useful.

Exercise 2. Suppose A is a square, m x m matrix and we have the SVD of the matrix A. Explain how you would efficiently solve the system Ax = b.

Exercise 3. Suppose we have an m x n matrix A with m > n and are solving a least squares problem with the normal equations. $A^TAx = A^Tb$. Suppose we have the singular value decomposition of A. Explain how you would efficiently solve this system.

Now let's see how we can find the singular value decomposition.

Exercise 4. For an m x n matrix A with SVD $U\Sigma V^T$, find A^TAv_i (where v_i is a column of V). Explain how we can find all the columns of V and the entries of Σ from the matrix A^TA. Now find Av_i for one of the columns of V. Explain how we can use this to find the columns of U.

Exercise 5. Write a Python function mySVD(A) that returns U, Sigma, and V for an m x n matrix A. You can use np.linalg.eig() to find the eigenvalues and eigenvectors of a matrix. Check it with np.linalg.svd()